

Filière : Génie Industriel 2

1

- le taux de distorsion harmonique du courant (THD),
- l'état de fonctionnement du moteur (normal / anormal).

Ces grandeurs permettent d'analyser le comportement thermique et électrique du moteur à l'aide de l'algorithme KNN.

Partie 1 : Régression

Cette partie introduit l'utilisation de l'algorithme KNN en mode **régression**. L'objectif est d'estimer la température interne du moteur, qui constitue une grandeur critique pour la fiabilité et la durée de vie de l'équipement.

La température interne dépend de phénomènes thermiques complexes et ne peut pas être déterminée par une relation analytique simple.

1. Chargement des bibliothèques et des données

Cette étape permet de charger les bibliothèques nécessaires ainsi que les données mesurées à partir d'un fichier CSV.

Code Python

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.neighbors import KNeighborsRegressor
5 from sklearn.preprocessing import MinMaxScaler
6 from sklearn.model_selection import train_test_split
7
8 data = pd.read_csv("moteur_data.csv")
9
10 Irms = data["Irms"].values
11 Tamb = data["Tamb"].values
12 Tmot = data["Tmot"].values
13 THD = data["THD"].values
14 Etat = data["Etat"].values
```

Explication ligne par ligne

- `import numpy as np` : importe la bibliothèque **NumPy**, utilisée pour le calcul numérique et la manipulation efficace de tableaux de données.
- `import pandas as pd` : importe **Pandas**, qui permet de lire et manipuler facilement les données issues d'un fichier CSV sous forme de tableaux structurés (DataFrame).

- `import matplotlib.pyplot as plt` : importe **Matplotlib**, utilisée pour la visualisation des données et des résultats (courbes, nuages de points, etc.).
- `from sklearn.neighbors import KNeighborsRegressor` : importe l'algorithme **KNN en mode régression**, destiné à estimer une grandeur continue.
- `from sklearn.preprocessing import MinMaxScaler` : importe un outil de **normalisation des données**, permettant de ramener toutes les variables dans un intervalle commun, généralement $[0, 1]$.
- `from sklearn.model_selection import train_test_split` : permet de diviser la base de données en un ensemble d'apprentissage et un ensemble de test.
- `pd.read_csv("moteur_data.csv")` : charge le fichier `moteur_data.csv` contenant les mesures du moteur électrique.

2. Analyse exploratoire des données

Avant d'appliquer l'algorithme KNN, il est important d'analyser visuellement les données afin de comprendre la relation entre le courant absorbé et la température interne du moteur.

Code Python

```
1 plt.figure(figsize=(6,4))
2 plt.scatter(Irms, Tmot)
3 plt.xlabel("Courant_RMS_(A)")
4 plt.ylabel("Temp rature_interne_( C )")
5 plt.grid(True)
6 plt.show()
```

Explication ligne par ligne

- `plt.figure(figsize=(6,4))` : crée une nouvelle figure avec une taille adaptée pour une bonne lisibilité.
- `plt.scatter(Irms, Tmot)` : trace un nuage de points représentant la relation entre le courant RMS et la température interne.
- `plt.xlabel(...)` : définit le nom de l'axe horizontal.
- `plt.ylabel(...)` : définit le nom de l'axe vertical.
- `plt.grid(True)` : ajoute une grille pour faciliter la lecture du graphique.
- `plt.show()` : affiche la figure à l'écran.

3. Normalisation des données

L'algorithme KNN étant basé sur des distances, il est indispensable de normaliser les variables afin d'éviter qu'une variable ayant une grande amplitude domine les autres.

Code Python

```
1 X = np.column_stack((Irms, Tamb))
2
3 scaler = MinMaxScaler()
4 Xn = scaler.fit_transform(X)
```

Explication ligne par ligne

- `np.column_stack((Irms, Tamb))` : regroupe les variables d'entrée dans une matrice où chaque ligne correspond à une observation.
- `scaler = MinMaxScaler()` : crée un objet de normalisation de type Min-Max.
- `fit_transform(X)` : ajuste le normaliseur aux données puis transforme les valeurs pour les ramener dans l'intervalle $[0, 1]$.

4. Application du KNN – Régression

Après la préparation des données, l'algorithme KNN est utilisé pour estimer la température interne du moteur à partir des grandeurs mesurées.

Code Python

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     Xn, Tmot, test_size=0.25, random_state=1)
3
4 knn = KNeighborsRegressor(n_neighbors=7)
5 knn.fit(X_train, y_train)
6 y_pred = knn.predict(X_test)
```

Explication ligne par ligne

- `train_test_split(...)` : sépare les données en un ensemble d'apprentissage (75%) et un ensemble de test (25%).
- `random_state=1` : garantit la reproductibilité du découpage.
- `KNeighborsRegressor(n_neighbors=7)` : crée un modèle KNN utilisant 7 voisins pour la régression.
- `fit(X_train, y_train)` : entraîne le modèle à partir des données d'apprentissage.
- `predict(X_test)` : estime la température interne pour les données de test.

5. Analyse des résultats – Régression

Cette étape permet d'évaluer visuellement la qualité de l'estimation obtenue par le modèle de régression.

Code Python

```
1 plt.figure(figsize=(6,4))
2 plt.scatter(y_test, y_pred)
3 plt.plot([Tmot.min(), Tmot.max()],
4          [Tmot.min(), Tmot.max()], 'r--')
5 plt.xlabel("Temp rature_r elle_( C )")
6 plt.ylabel("Temp rature_estim e_( C )")
7 plt.grid(True)
8 plt.show()
```

Explication ligne par ligne

- `plt.scatter(y_test, y_pred)` : compare les valeurs réelles et estimées sous forme de nuage de points.
- La droite en pointillés rouges représente le cas idéal où la température estimée est égale à la température réelle.
- Plus les points sont proches de la diagonale, plus l'estimation est précise.

Partie 2 : Classification

Dans cette partie, l'algorithme KNN est utilisé en mode classification afin de détecter un fonctionnement normal ou anormal du moteur à partir de signatures électriques.

1. Application du KNN – Classification

Code Python

```
1 Xc = np.column_stack((Irms, THD))
2 scaler = MinMaxScaler()
3 Xc_n = scaler.fit_transform(Xc)
4 X_train, X_test, y_train, y_test = train_test_split(
5     Xc_n, Etat, test_size=0.25, random_state=2)
6 from sklearn.neighbors import KNeighborsClassifier
7 knn = KNeighborsClassifier(n_neighbors=5)
8 knn.fit(X_train, y_train)
9 y_pred = knn.predict(X_test)
```

Explication ligne par ligne

- Irms et THD constituent les variables d'entrée de la classification.
- La normalisation est appliquée pour garantir un calcul de distance équilibré.
- Etat représente la classe à prédire (normal ou anormal).
- `KNeighborsClassifier(n_neighbors=5)` : crée un classifieur KNN basé sur 5 voisins.
- `predict` : attribue une classe à chaque observation de test.

2. Analyse des résultats – Classification

Dans cette partie du TP, nous analysons les résultats de la classification des états de fonctionnement du système électrique à partir de deux indicateurs importants : le courant efficace (Irms) et le taux de distorsion harmonique (THD). L'objectif est de visualiser la capacité de ces paramètres à discriminer les états normaux et anormaux du système.

Code Python

```
1 plt.figure(figsize=(6,4))
2 plt.scatter(Irms[Etat==0], THD[Etat==0], label="Normal")
3 plt.scatter(Irms[Etat==1], THD[Etat==1], label="Anormal")
4 plt.xlabel("Courant_RMS_(A)")
5 plt.ylabel("THD_(%)")
6 plt.legend()
7 plt.grid(True)
8 plt.show()
```

Explication ligne par ligne

- Chaque point représente une condition de fonctionnement du moteur.
- Les couleurs distinguent les états normal et anormal.
- Les zones de recouvrement illustrent les limites du classifieur KNN.

Exercice 2

Dans cet exercice, nous nous intéressons à l'utilisation de l'algorithme K-Nearest Neighbors (KNN) dans le contexte des bâtiments intelligents. À partir de données issues de capteurs environnementaux (température, humidité, lumière, CO2), l'objectif est d'aborder deux problématiques complémentaires du machine learning. La première consiste à résoudre un problème de classification, visant à détecter l'état d'occupation d'un bureau (occupé ou non occupé). La seconde porte sur un problème de régression,

dont le but est de prédire la température intérieure du bâtiment à partir des mêmes variables explicatives.

Travail demandé

1. Charger le jeu de données fourni.
2. Visualiser les variables.
3. Implémenter un modèle KNN pour estimer l'occupation.
4. Évaluer les performances à l'aide de l'accuracy et de la matrice de confusion.
5. Implémenter un modèle KNN pour prédire la température interieure.
6. Évaluer les performances à l'aide de RMSE.